TALENT ACQUISITION SYSTEM (TAS)

A project report submitted to ICT Academy of Kerala

in partial fulfillment of the requirements

for the certification of

ADVANCED

PYTHON

Submitted by

S A SANUSH

SADARSH S

ABIN FRANCIS

ARSHA P V

ARATHY KRISHNA

ICT ACADEMY OF KERALA

THIRUVANANTHAPURAM, KERALA, INDIA

May 2026

LIST OF FIGURES

Figure No.	Title
Figure 1	Home page of the Talent Acquisition System
Figure 2	Proposed system architecture of the Talent Acquisition System
Figure 3	SQLite database schema used for users, reports and contacts
Figure 4	System architecture of the Talent Acquisition System
Figure 5	Resume upload interface
Figure 6	Resume analysis result with score breakdown
Figure 7	User scan history and saved report details
Figure 8	Admin dashboard for catalog and message management
Figure 9	Criteria-wise score visualization generated by the application

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
ATS	Applicant Tracking System
CRUD	Create, Read, Update and Delete
CSV	Comma Separated Values
CSRF	Cross-Site Request Forgery
DBMS	Database Management System
DOCX	Microsoft Word Open XML Document
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
TAS	Talent Acquisition System
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

Abstract **6**

1. Problem Definition **7**

1.1 Overview 7

1.2 Problem Statement 8

2. Introduction **9**

2.1 Background Of The Project 9

2.2 Objectives Of The Project 10

2.3 Scope Of The Project 10

2.4 Methodology 11

3. Literature Survey **12**

3.1 Review Of Resume Screening Methods 12

3.2 Review Of Applicant Tracking Systems 13

3.3 Review Of NLP In Resume Analysis 13

3.4 Review Of Job Recommendation Techniques 14

4. System Analysis **15**

4.1 Existing System 15

4.2 Limitations Of Existing System 16

4.3 Proposed System 16

4.4 Feasibility Study 17

4.5 Functional Requirements 17

4.6 Non-Functional Requirements 17

5. System Design **19**

5.1 System Architecture 20

5.2 Data Flow Diagram Description 20

5.3 Database Design 21

5.4 User Interface Design 21

5.5 Security Design 21
5.6 Module Design 22
6. Implementation 23
6.1 Technology Stack And Project Structure 24
6.2 Resume Upload And Text Extraction 26
6.3 Resume Analysis, Scoring And Matching 27
6.4 Report Storage, History And Pdf Export 29
6.5 Admin, Contact And Validation Modules 31
6.6 Testing Strategy And Test Case Summary 33
7. Result 35
7.1 Sample Output Description 36
7.2 Performance Observation 36
7.3 Advantages Of The System 37
7.4 Limitations 37
7.5 Future Enhancements 37
8. Conclusion 39
8.1 Summary Of Work Completed 39
8.2 Learning Outcomes 39
Appendix A: Project File Structure 41
Appendix B: Important Routes 42
Appendix C: Important Functions 43
Appendix D: Requirements 44
Appendix E: Important Code Snippets 45
References 47

ABSTRACT

The Talent Acquisition System is a web-based resume analysis and job matching application developed using Python, Flask, SQLite and supporting data-processing libraries. The project is designed to help students, job seekers, placement officers and training teams evaluate resumes in a structured manner before applying for jobs or forwarding candidate profiles to recruiters.

The system accepts resumes in PDF, DOCX and TXT formats, validates the uploaded file, extracts readable text, analyses the content and generates a score based on important resume-quality criteria. The criteria include contact information, professional summary, work experience, education, skills, talent acquisition optimization, consistency, proofreading, file format and relevance to a target role. Users may also enter a target job title and job description so that the system can compare the resume with role-specific requirements.

The application provides job recommendations, matched keywords, missing keywords, category-wise score explanations, resume improvement suggestions and locally generated bullet rewrite ideas. Registered users can save scan history, revisit previous reports and export report details as PDF. An administrator can view platform metrics, review contact messages and add new roles to the recommendation catalog.

The project demonstrates concepts from data science and analytics by converting unstructured resume text into structured indicators, applying rule-based feature extraction, using weighted scoring models, performing keyword matching and presenting interpretable results. It also demonstrates practical software engineering through secure upload handling, password hashing, CSRF protection, SQLite persistence, Jinja templates and pytest-based automated testing.

1. PROBLEM DEFINITION

Resume screening is one of the earliest and most important stages in recruitment. A resume often decides whether a candidate receives an interview opportunity, yet many candidates do not know whether their resume contains enough role-specific information, measurable achievements, keywords, education details or professional formatting. Manual checking is possible, but it is slow, inconsistent and difficult to scale in a training or placement environment.

The project defines the problem as the need for a local, transparent and repeatable resume evaluation system that can analyse candidate resumes, generate meaningful scores, identify missing areas and provide job-role recommendations. The solution must be simple enough for students and placement teams to use, but detailed enough to explain why a resume received a particular score.

1.1 OVERVIEW

In educational institutions, placement cells and recruitment support programmes, resume review is usually handled by mentors, trainers or placement officers. This approach helps candidates, but the quality and speed of review depend heavily on the reviewer. When many students submit resumes at the same time, manual review becomes repetitive and delays feedback.

Modern recruitment also uses keyword-based screening and applicant tracking systems. If a resume does not clearly mention relevant tools, skills, achievements and job-specific terms, it may perform poorly even when the candidate is capable. The Talent Acquisition System addresses this issue by giving candidates an immediate analysis of their resume and by showing how closely it matches selected job roles.

The proposed system is implemented as a Flask web application. It provides resume upload, text extraction, scoring, target role matching, job recommendation, scan history, PDF export, contact support and an admin dashboard. The system stores reports in SQLite and keeps uploaded resumes temporary, reducing unnecessary storage of private files.

1.2 PROBLEM STATEMENT

Manual resume evaluation is time-consuming, subjective and hard to maintain at scale. Candidates may not receive timely feedback, reviewers may apply different standards and institutions may not have a reliable way to preserve analysis history. Existing generic resume tools may not be transparent about scoring logic and may not support local placement-focused role catalogs.

The problem is to design and implement a web application that can accept resumes, validate file safety, extract text, evaluate resume quality using defined criteria, compare the resume with target job requirements, recommend suitable roles, generate improvement suggestions, store scan results for authenticated users and provide administrative controls for maintaining the recommendation catalog.

The system must support PDF, DOCX and TXT uploads, calculate a score out of 100, show category-wise scoring, identify matched and missing keywords, allow report history and PDF export, provide secure login and registration, and include automated tests to verify important workflows.

2. INTRODUCTION

Talent acquisition has become increasingly data-oriented. Recruiters and placement teams do not only read resumes for general presentation; they also look for measurable achievements, relevant technical skills, role-specific keywords and evidence of communication ability. A candidate who understands this expectation can prepare a more effective resume.

The Talent Acquisition System is built to support this preparation process. The application allows the user to upload a resume and optionally enter a target job title and job description. The backend extracts the resume text, normalizes it, detects important sections and evaluates the document using a weighted scoring model. The final output is not only a score, but also an explanation of strengths, weaknesses and suggested improvements.

The system is divided into user-facing and administrator-facing features. User-facing features include resume upload, result display, score breakdown, job matching, missing keyword analysis, bullet rewrites, scan history and report export. Administrator-facing features include dashboard metrics, contact message review and job catalog updates.

From a data science perspective, the project works with unstructured text. It converts resume content into structured features such as detected contact details, extracted skills, education indicators, estimated experience, action verbs, keyword matches and relevance scores. These indicators are then combined through weighted rules to produce interpretable analytics.

From a software development perspective, the project uses Flask for routing and rendering, SQLite for persistence, pandas for job catalog handling, NumPy for numeric handling, pypdf and python-docx for text extraction, optional OCR libraries for scanned documents, Werkzeug security utilities for password hashing and safe filenames, and pytest for automated testing.

2.1 BACKGROUND OF THE PROJECT

The background of the project is connected with the growing importance of resume quality in recruitment and placement activities. Students and fresh graduates often possess useful technical knowledge, but their resumes may not communicate that knowledge clearly. A resume that lacks relevant keywords, project descriptions, tools, measurable outcomes or professional structure may fail in the first level of screening.

The Talent Acquisition System was planned as a practical academic project that connects software development with data analytics. Instead of only storing candidate details, the system analyses resume text and converts it into meaningful signals. These signals are then used to produce a score, show missing keywords and recommend roles that match the candidate profile.

The project is also useful in training environments because it gives immediate feedback. A student can upload a resume, read the suggestions, improve the document and scan it again. This repeatable feedback loop supports learning and helps candidates understand how small improvements can increase resume quality.

2.2 OBJECTIVES OF THE PROJECT

The primary objective of the project is to build a web application that can evaluate resumes in a structured, transparent and user-friendly way. The system should reduce manual effort for initial resume checking and provide useful feedback to candidates.

The project objectives include resume upload support, text extraction from common document formats, rule-based analysis, score calculation, target job matching, job recommendation, report storage, PDF export, user authentication and admin-side catalog maintenance. These objectives together make the application a complete mini talent acquisition support system.

Another important objective is explainability. The user should not receive only a final number. The system should show why the score was assigned, which areas are strong, which areas require improvement and which keywords are missing for a selected role.

2.3 SCOPE OF THE PROJECT

The project scope covers resume readiness analysis and simple job role recommendation. It supports three file formats: PDF, DOCX and TXT. It allows a user to enter a target job title and job description, and it compares the resume content with this target text.

The system stores analysis reports for logged-in users and allows them to view history. This makes it possible to compare resume improvements over time. The admin module supports contact message review and job catalog updates, which helps the application remain useful for different placement requirements.

The project does not aim to replace a full enterprise applicant tracking system. It does not perform final hiring decisions, interview scheduling or recruiter workflow automation. Its scope is focused on resume analysis, feedback generation and role matching for academic and small-scale placement use.

2.4 METHODOLOGY

The methodology followed in the project begins with identifying the resume review problem and defining the expected users. The next step is selecting a suitable technology stack. Python and Flask were selected because they are simple, flexible and suitable for rapid web application development.

After the technology stack was selected, the system was divided into modules such as upload handling, text extraction, scoring, matching, database storage, authentication, admin control and report export. Each module was implemented using functions in app.py and connected to the user interface through Flask routes.

Testing was performed using pytest. The test cases cover important workflows such as file validation, CSRF protection, text resume analysis, mismatched file rejection, authentication, history access, PDF export and admin catalog updates. This methodology supports both implementation and verification.

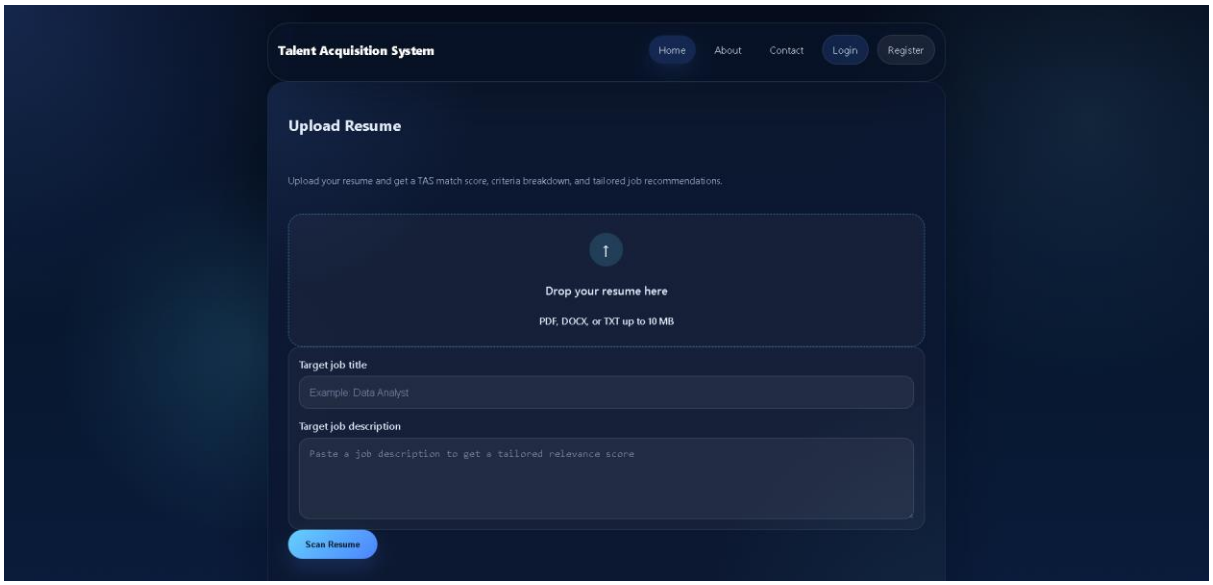


Figure 1: Home page of the Talent Acquisition System

3. LITERATURE SURVEY

Resume screening and applicant tracking have been widely studied in recruitment technology. Traditional screening depends on human judgment, where recruiters manually read resumes and compare them with job requirements. This method is flexible and context-aware, but it can become slow and inconsistent when the number of applicants is high.

Applicant Tracking Systems introduced structured digital screening. These systems store applications, filter candidates and help recruiters search for skills or keywords. However, many candidates struggle because their resumes are not written in a way that clearly matches job descriptions. Research and industry practice both show that resume relevance, keyword alignment and clarity of achievements strongly influence shortlisting decisions.

Natural Language Processing is commonly used to extract information from resumes. Basic techniques include tokenization, keyword extraction, section detection and similarity comparison. Advanced systems may use named entity recognition, semantic embeddings or machine learning classifiers. For an educational project, a transparent rule-based model is appropriate because users can understand how the score is calculated and why suggestions are produced.

Job recommendation systems often compare candidate profiles with role requirements. Similarity can be calculated using keyword overlap, weighted terms or vector-based measures. The Talent Acquisition System uses a practical combination of skill synonym matching, role keyword inference, weighted terms and cosine-style similarity. This makes the system explainable and easy to maintain.

Security is also an important concern in resume processing systems because resumes contain personal data. Good practice includes validating uploaded files, limiting upload size, avoiding permanent storage of raw resumes, hashing passwords, protecting forms from CSRF attacks and restricting administrative features. These practices are reflected in the project implementation.

3.1 REVIEW OF RESUME SCREENING METHODS

Resume screening methods can be divided into manual screening, checklist-based screening and automated screening. Manual screening allows human understanding but consumes time. Checklist-based screening improves consistency but still requires human effort. Automated

screening can process documents quickly but must be designed carefully to avoid unclear or unfair results.

The project adopts a transparent automated approach. It uses fixed criteria and clear scoring rules, so the candidate can understand the result. The system checks whether the resume contains basic professional details and whether it aligns with the target role.

This approach is suitable for an academic project because it demonstrates analytics without hiding the logic inside a complex model. A future version can add machine learning or semantic models, but the current implementation prioritizes clarity, maintainability and explainable feedback.

3.2 REVIEW OF APPLICANT TRACKING SYSTEMS

Applicant Tracking Systems are widely used by organizations to manage large numbers of job applications. They help recruiters collect resumes, search candidate profiles and filter applications based on job requirements. These systems are useful, but candidates often do not know how their resumes are interpreted by such tools.

The Talent Acquisition System uses the idea of ATS-style screening in a simplified educational form. It detects resume sections, checks keywords and calculates a score. The goal is to help candidates prepare resumes that are easier for recruitment systems and human reviewers to understand.

Unlike many commercial tools, this project keeps the scoring process visible. The user can see category-wise scores, matched keywords and missing keywords. This makes the application more suitable for learning and placement preparation.

3.3 REVIEW OF NLP IN RESUME ANALYSIS

Natural Language Processing helps computers process human language. In resume analysis, NLP can be used for text extraction, tokenization, keyword detection, named entity recognition, skill extraction, experience estimation and similarity comparison.

The project uses practical NLP-inspired techniques such as text normalization, regular expressions, keyword matching and skill synonym detection. Optional spaCy support is included so that the application can use NLP features when the library is available in the local environment.

The system avoids dependence on a heavy external model for core operation. This decision improves portability because the app can still work even if a full spaCy model is not installed. It also makes the logic easier to explain in an academic report.

3.4 REVIEW OF JOB RECOMMENDATION TECHNIQUES

Job recommendation systems compare a candidate profile with role requirements. Some systems use keyword overlap, some use statistical similarity and advanced systems use machine learning models trained on large datasets. For this project, a lightweight and interpretable method is used.

The application stores job roles in a CSV catalog. Each role contains a title, keywords and a description. The resume text is compared with these catalog entries, and the system identifies roles that appear relevant to the candidate's detected skills and experience.

The target job matching feature improves recommendation quality because it allows the user to provide role-specific input. If the user enters a Data Analyst job description, the system can highlight missing analytics keywords rather than giving only generic resume advice.

4. SYSTEM ANALYSIS

The existing resume review process is mostly manual. Candidates send resumes to mentors or placement officers, and feedback is returned through meetings, email or messaging platforms. Manual review can identify nuanced problems, but it is difficult to provide fast feedback to many users. It also does not automatically store previous scan results or compare different versions of a resume.

The proposed system automates the first level of resume review. It does not replace human recruiters; instead, it provides structured preliminary analysis. It helps users identify missing sections, weak keyword alignment, insufficient skills, lack of measurable achievements and formatting issues before the resume reaches a recruiter.

Functional requirements include user registration, login, logout, resume upload, file validation, text extraction, scoring, job matching, result display, report saving, history viewing, PDF export, contact form submission and admin catalog management. Non-functional requirements include usability, reliability, security, maintainability, performance for small to medium resumes and clarity of output.

The hardware requirements are modest because the application is a lightweight Flask project. A standard laptop or desktop with Python installed is sufficient. The software requirements include Python, Flask, SQLite, pandas, NumPy, pypdf, python-docx, spaCy support, pytest and optional OCR dependencies such as pytesseract and pdf2image.

The feasibility of the project is high. Technical feasibility is supported by widely used open-source Python libraries. Operational feasibility is strong because users interact through a browser and do not need to understand the backend. Economic feasibility is also favorable because SQLite and the selected Python libraries are free for educational use.

4.1 EXISTING SYSTEM

The existing system in many institutions is manual resume checking. Students submit resumes to mentors or placement officers, and feedback is given after review. This process is familiar and flexible, but it is not efficient when many students require feedback at the same time.

Manual review also depends on reviewer experience. One reviewer may focus on formatting, another may focus on technical skills and another may focus on grammar. Because of this,

two candidates may receive feedback of different depth even if their resumes have similar issues.

Existing generic online resume analyzers may be available, but they may not support institution-specific placement needs, local job catalogs or saved history. They may also provide scores without explaining the reasoning clearly.

4.2 LIMITATIONS OF EXISTING SYSTEM

The limitations of the existing system include slow feedback, inconsistent evaluation, limited history tracking and difficulty in comparing a resume with a target job description. Manual review also becomes repetitive because many resumes have common issues such as missing skills, weak summaries and unclear project descriptions.

Another limitation is the absence of measurable output. A student may receive general comments such as improve formatting or add more keywords, but may not know which specific categories need attention. The absence of a score breakdown makes improvement less focused.

The existing process may also lack a central database. Without stored reports, it is difficult to track whether a resume has improved after revision. The Talent Acquisition System addresses these limitations through automated scoring, history and structured report generation.

4.3 PROPOSED SYSTEM

The proposed system is a Flask-based web application that accepts resumes, extracts text, analyses content and returns detailed feedback. It provides immediate results to the user and stores reports for logged-in users.

The proposed system includes modules for authentication, upload validation, text extraction, scoring, role matching, recommendation, report storage, contact management and admin control. These modules work together to provide a complete resume analysis workflow.

The system is designed for simplicity and transparency. Instead of using a black-box prediction model, it uses clear criteria and weighted scoring. This makes the results easier to justify during project evaluation and easier for students to understand.

4.4 FEASIBILITY STUDY

Technical feasibility is strong because the project uses stable and widely available tools. Flask provides web routing, SQLite provides lightweight persistence, pandas supports CSV handling and Python libraries provide document parsing. The application can run on a normal development machine.

Operational feasibility is also strong because the system is browser-based. Users do not need to install a separate desktop application. They can upload a resume, enter optional job details and view results from a simple interface.

Economic feasibility is high because the project uses free and open-source technologies. It does not require paid cloud services or enterprise database servers for demonstration. The main requirement is a machine with Python and the listed dependencies installed.

4.5 FUNCTIONAL REQUIREMENTS

The functional requirements define what the system should do. The system should allow users to register, log in, log out, upload resumes, enter target role details, view analysis results, view scan history and export saved reports as PDF.

The system should validate file extensions and check whether the file content matches the extension. It should extract text from PDF, DOCX and TXT files. It should calculate a score, generate category-wise explanations, show matched jobs and list suggestions.

The admin should be able to access an admin dashboard, view contact messages and add job roles to the recommendation catalog. The system should also store contact form submissions and validate email input before saving messages.

4.6 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality expectations of the system. The application should be reliable, simple to use, maintainable and secure enough for local academic use. It should respond quickly for normal resume files and should give clear error messages when input is invalid.

The system should protect user accounts through password hashing and should protect form submissions using CSRF tokens. It should limit upload size and remove temporary resume files after processing. These controls improve privacy and reduce misuse.

Maintainability is achieved by keeping related logic inside reusable helper functions. Testing improves confidence during future changes. The use of SQLite and CSV also keeps the project easy to understand and demonstrate.

5. SYSTEM DESIGN

The system follows a simple web application architecture. The browser sends HTTP requests to the Flask server. Flask routes handle user actions, call helper functions for validation and analysis, interact with SQLite for persistence and render HTML templates using Jinja2. Static files provide styling, client-side interaction and visual polish.

The main workflow begins when a user submits a resume through the upload form. The server validates the extension, saves the file with a safe temporary filename, checks file signatures, extracts text based on file type and deletes the temporary upload after processing. The extracted text is normalized and passed through the analysis pipeline.

The analysis pipeline builds a resume profile by detecting skills, action verbs, experience, education signals and other indicators. It then calculates criteria-wise scores, builds score explanations, applies role-adjusted weights when a target title is provided and generates an overall score out of 100. Job recommendations are created by comparing resume content with the local job catalog and inferred target-role requirements.

The database design contains three main tables. The users table stores usernames and hashed passwords. The reports table stores report ownership, filename, timestamp, score, matched jobs, criteria, suggestions and analysis data as JSON text. The contacts table stores contact form submissions with name, email, subject, message and timestamp.

Security design includes password hashing through Werkzeug, session-based authentication, admin access control, CSRF token checks, rate limiting for sensitive routes, upload size limits, extension validation, file signature checks, secure filenames and temporary upload cleanup. These controls are important because resumes and user accounts may contain sensitive information.

User Browser	**Flask Routes**	**Jinja Templates**
Resume Upload	**Analysis Engine**	**Result Dashboard**
Job Catalog CSV	**SQLite Database**	**PDF Export**

Figure 2: Proposed system architecture of the Talent Acquisition System

Upload Resume -> Validate File -> Extract Text -> Analyse Profile -> Score Criteria -> Match Jobs -> Save Report -> Display Result

users	reports	contacts
id	id	id
username	user_id	user_id
password_hash	filename	name
created_at	uploaded_at	email
	score	subject
	matched_jobs	message
	criteria	created_at
	suggestions	
	analysis	

Figure 3: SQLite database schema used for users, reports and contacts

5.1 SYSTEM ARCHITECTURE

The system architecture follows a client-server pattern. The client is a web browser, and the server is the Flask application. The browser sends requests to routes such as home, analyze, login, register, history, contact and admin. The server processes these requests and returns rendered HTML pages.

The backend uses helper functions to separate responsibilities. File handling functions manage uploads, extraction functions read documents, analysis functions calculate scores, database functions save and retrieve data, and route functions connect user actions to backend processing.

The storage layer contains SQLite for user accounts, reports and contact messages. The job catalog is stored in a CSV file because it is simple to update and suitable for a small recommendation list.

5.2 DATA FLOW DIAGRAM DESCRIPTION

The data flow begins when the user selects a resume file and submits the upload form. The file, target title and target job description are sent to the Flask server. The server validates the request and temporarily stores the uploaded file.

After validation, the system extracts text from the file. The extracted text is normalized and analysed. The analysis result contains criteria scores, profile details, matched jobs, missing keywords, suggestions and bullet rewrite ideas.

The report is saved into the database if the workflow completes successfully. The result is then rendered on the result page. For logged-in users, the same report can later be accessed from the history page and exported as a PDF.

5.3 DATABASE DESIGN

The database design is intentionally simple. SQLite is used because the project is a local web application and does not require a separate database server. The database file is named tas_reports.db and is created inside the project directory.

The users table stores user account details. The password is stored as a hash, not as plain text. The reports table stores resume analysis data. Complex fields such as matched jobs, criteria, suggestions and analysis are stored as JSON text. The contacts table stores messages submitted through the contact page.

This design supports the current requirements while keeping the project understandable. The use of JSON inside SQLite provides flexibility for report structures without creating a large number of relational tables.

5.4 USER INTERFACE DESIGN

The user interface is designed around the main workflow of resume scanning. The home page provides the upload form and target job inputs. The result page displays the score, explanation, job matches and suggestions in an organized manner.

The history page helps registered users review previous scans. The detail page shows saved report information and provides export functionality. The login and registration pages support account access, while the contact page allows users to send messages.

The admin page gives the administrator a focused view of platform activity and catalog management. The UI uses templates, CSS and JavaScript to keep the experience consistent across pages.

5.5 SECURITY DESIGN

Security design is important because resumes may contain personal contact details, education information and work history. The application avoids permanent storage of uploaded resume files. Files are saved temporarily for extraction and then removed after processing.

The system uses secure_filename to sanitize uploaded names and UUID-based temporary filenames to avoid conflicts. It checks file extensions and file signatures to reduce the risk of incorrectly labelled files. Upload size is limited to prevent very large files from affecting the application.

User passwords are hashed using Werkzeug. CSRF protection is applied to form submissions, and admin routes are restricted to the configured admin user. These measures make the project safer for demonstration and local use.

5.6 MODULE DESIGN

The application can be understood as a group of modules. The authentication module handles registration, login, logout and current user detection. The upload module handles file validation and temporary storage. The extraction module reads text from supported resume formats.

The analysis module detects resume features and calculates scores. The matching module compares resume content with target roles and catalog roles. The reporting module stores reports, retrieves history and exports PDF files. The admin module manages messages and job catalog entries.

Although these modules are implemented in a single app.py file, the functions are logically separated. This organization makes it easier to explain, test and improve the project.

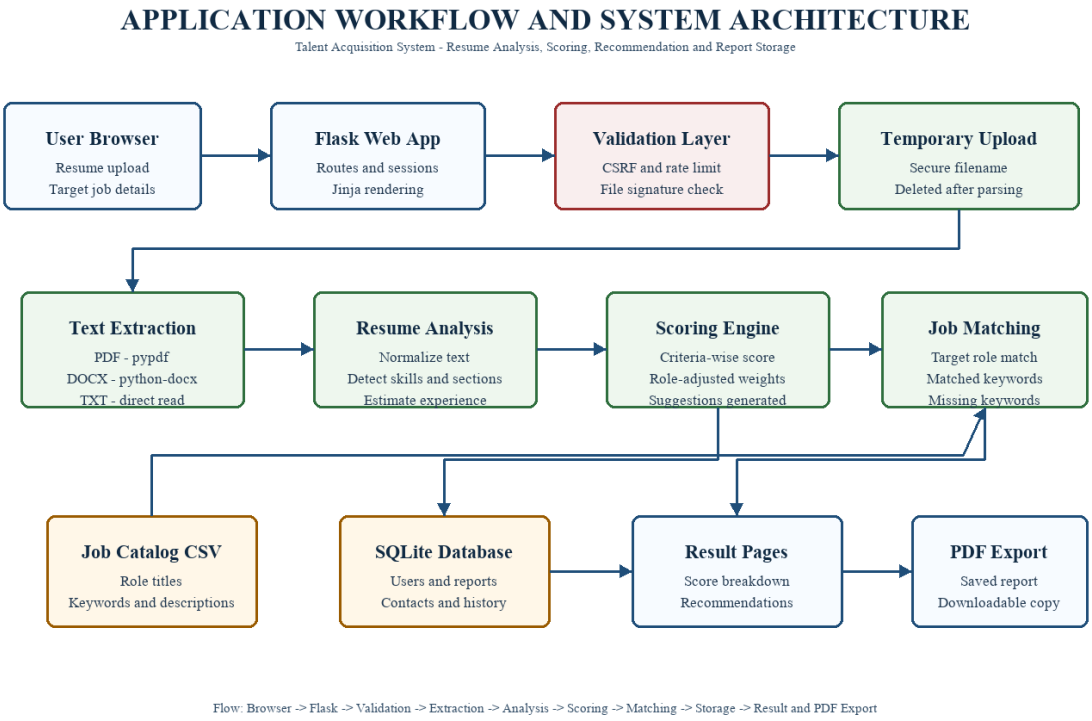


Figure 4: System architecture of the Talent Acquisition System

6. IMPLEMENTATION

The project is implemented mainly in app.py. This file initializes the Flask application, configures upload and chart folders, defines constants, loads optional NLP and OCR dependencies, initializes SQLite tables, defines helper functions and implements route handlers.

Resume file handling is implemented through allowed_file, save_upload_temporarily, file_signature_matches and extract_resume_text. PDF extraction uses pypdf, DOCX extraction uses python-docx and TXT files are read directly. Optional OCR support can process scanned PDFs when pytesseract and pdf2image are available in the local environment.

Resume analysis is implemented through normalization, keyword detection, skill synonym matching, experience estimation, action verb detection, criteria scoring and score explanations. The criteria scoring functions evaluate contact information, professional summary, experience, education, skills, TAS optimization, consistency, proofreading, file format and relevance.

Role matching is implemented through target job matching, role keyword inference, weighted term comparison and job catalog matching. The built-in catalog is stored in job_data.csv and contains example roles such as Data Analyst, Software Engineer, Product Manager, Technical Writer and Digital Marketing Specialist. Admin users can add catalog roles from the dashboard.

Report persistence is implemented using SQLite. Nested report details such as criteria scores, matched jobs, suggestions, analysis steps and bullet rewrites are stored as JSON text. This keeps the schema simple while still preserving rich report information.

The user interface is built with Jinja templates in the templates folder. The application includes pages for the home upload form, result view, history dashboard, report detail, login, registration, admin dashboard, contact page and about page. CSS and JavaScript files in the static folder provide styling and frontend interaction.

Testing is implemented with pytest. The test suite verifies file validation, target job matching, skill alias extraction, experience estimation, role-adjusted scoring, CSRF protection, resume analysis, temporary upload deletion, bullet rewrite generation, mismatched file rejection, contact validation, registration, login, history, PDF export and admin catalog updates.

6.1 TECHNOLOGY STACK AND PROJECT STRUCTURE

The technology stack includes Python, Flask, SQLite, pandas, NumPy, pypdf, python-docx, spaCy support, pytesseract support, pdf2image support, HTML, CSS, JavaScript and pytest. Each technology has a specific role in the project.

Python is used as the primary programming language. Flask handles the web application layer. SQLite stores persistent data. pandas reads and updates the job catalog. NumPy supports score calculations. pypdf and python-docx extract text from resume files.

The frontend is built using HTML templates, CSS styling and JavaScript behavior. pytest is used to verify application workflows. This stack is appropriate for a data science and analytics project because it combines web development with text analytics.

The main file is app.py. It contains application configuration, database initialization, helper functions, analysis logic and route definitions. The templates folder contains Jinja2 HTML pages used by Flask to render the user interface.

The static folder contains CSS, JavaScript and images. The job_data.csv file stores the recommendation catalog. The tas_reports.db file stores users, reports and contact messages during runtime. The tests folder contains pytest test cases.

The reports folder stores documentation outputs such as this project report. The requirements.txt file lists Python dependencies needed to run the application.

Technology	Description
Python	Primary programming language used for backend logic, text processing, scoring and application control.
Flask	Lightweight web framework used for routing, request handling, sessions and page rendering.
SQLite	File-based relational database used to store users, reports and contact messages.
HTML	Markup language used to structure the web pages and form interfaces.
CSS	Styling language used to design layouts, score cards, dashboards and responsive pages.
JavaScript	Client-side scripting used for interactive behavior and frontend enhancements.
Jinja2	Template engine used by Flask to render dynamic HTML pages with backend data.
pandas	Data handling library used to load and update the job recommendation catalog.
NumPy	Numerical library used for score clipping and reliable numeric calculations.
pypdf	PDF processing library used to extract readable text from uploaded PDF resumes.

python-docx	Document processing library used to extract text from DOCX resume files.
spaCy	Optional NLP library used to support natural language processing features when available.
pytesseract	Optional OCR wrapper used to read scanned resume text when OCR support is configured.
pdf2image	Optional utility used to convert scanned PDF pages into images for OCR processing.
Werkzeug	Security utility library used for password hashing and safe uploaded filenames.
pytest	Testing framework used to verify validation, analysis, authentication and admin workflows.

Table 1: Technologies used in the Talent Acquisition System

6.2 RESUME UPLOAD AND TEXT EXTRACTION

Resume upload is handled through a form on the home page. When the user submits the form, the backend checks whether a file was provided and whether the extension is allowed. Supported extensions are PDF, DOCX and TXT.

The uploaded file is saved temporarily using a secure generated filename. This reduces the risk of filename conflicts and unsafe paths. After saving, the system checks whether the file signature matches the declared extension.

After the text is extracted, the temporary file is deleted. This is an important implementation decision because it avoids storing raw resumes permanently and supports better privacy.

Text extraction is handled based on file type. For PDF files, the system uses pypdf to read pages and extract text. For DOCX files, python-docx is used to read paragraph content. For TXT files, the file content is read directly.

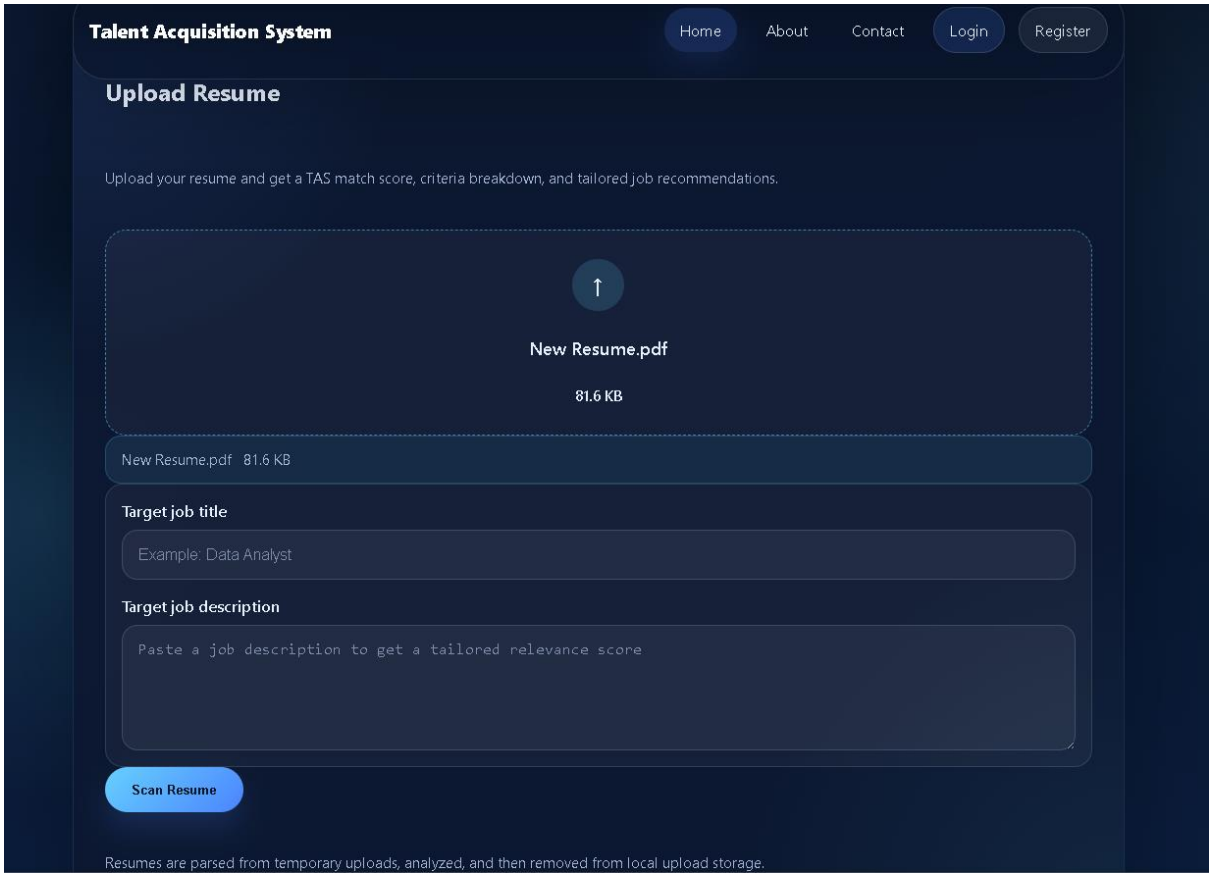


Figure 5: Resume upload interface

Optional OCR support is included for scanned PDFs. OCR requires pytesseract and pdf2image along with native system dependencies. Because these dependencies may not always be installed, OCR is optional and the main application can run without it.

The extracted text is normalized before analysis. Normalization reduces irregular spacing and makes matching more consistent. This step is important because resume documents can contain different formatting styles.

6.3 RESUME ANALYSIS, SCORING AND MATCHING

Resume profile extraction converts raw resume text into structured information. The system detects contact information, skills, action verbs, possible experience indicators and education signals. These details help the application understand the resume beyond simple word count.

Skill extraction uses a synonym dictionary. For example, SQL can be detected through related terms such as database, MySQL or PostgreSQL. Data visualization can be detected through terms such as Tableau, Power BI, matplotlib or dashboards.

The profile extraction process supports both scoring and recommendation. Detected skills can increase the skills score and can also help identify suitable job roles from the catalog.

The scoring model evaluates the resume using multiple criteria. The criteria include contact information, professional summary, work experience, education, skills, TAS optimization, consistency, proofreading, file format and relevance. Each criterion receives a score and explanation.

The final score is calculated using weighted scoring. Skills and work experience carry higher importance because they directly affect job readiness. Contact information, education, formatting and proofreading also contribute to the final score.

The system supports role-adjusted weights. For example, when the target role is Data Analyst, the system can give greater importance to skills and relevance. This makes the score more meaningful for the selected job context.

Target job matching allows the user to enter a job title and job description. The system extracts important terms from this target input and compares them with the resume text. It then identifies matched and missing keywords.

This feature is important because a general resume score may not be enough. A resume can be well-written but still poorly aligned with a specific job. Target matching helps the user understand what to add or emphasize for a selected role.

The system also infers role keywords from known job titles. For example, a Data Analyst role is associated with Python, SQL, pandas, dashboards, reporting and communication. These hints improve matching even when the user enters only a job title.

Job recommendation is based on the local job catalog stored in job_data.csv. Each catalog entry contains a title, keywords and description. The system compares resume content with these entries and returns applicable roles.

The recommendation process considers detected skills, keyword matches and target job information. This allows the system to suggest roles that are reasonably connected to the candidate's resume.

The admin can add new roles to the catalog, making the recommendation system adaptable. For example, if a placement batch focuses on data engineering roles, the admin can add a Data Engineer role with relevant keywords.

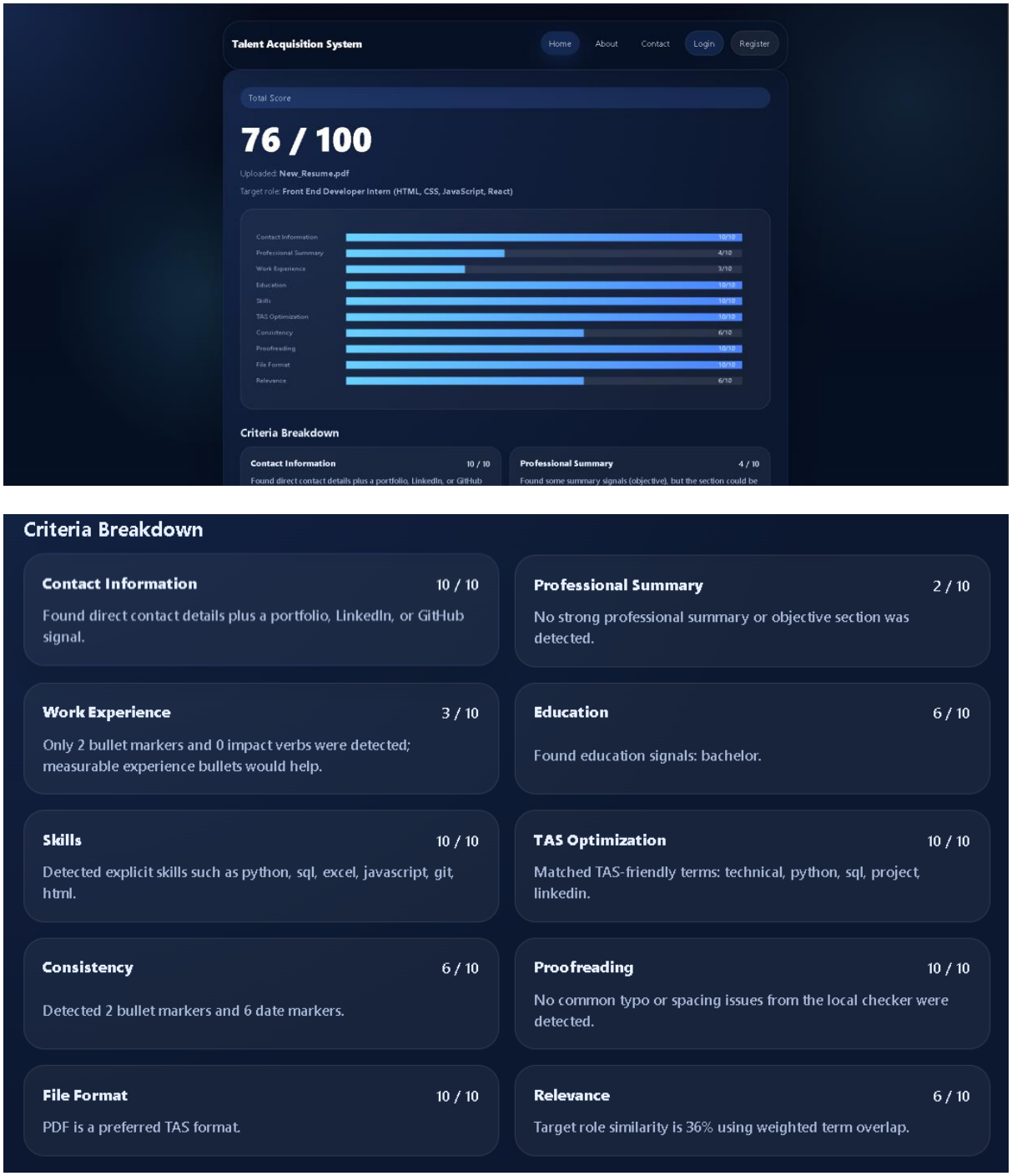


Figure 6: Resume analysis result with score breakdown

6.4 REPORT STORAGE, HISTORY AND PDF EXPORT

When a resume scan is completed, the report is stored in SQLite. The stored data includes filename, upload time, score, matched jobs, criteria, suggestions and detailed analysis information. Logged-in users can later access these saved reports.

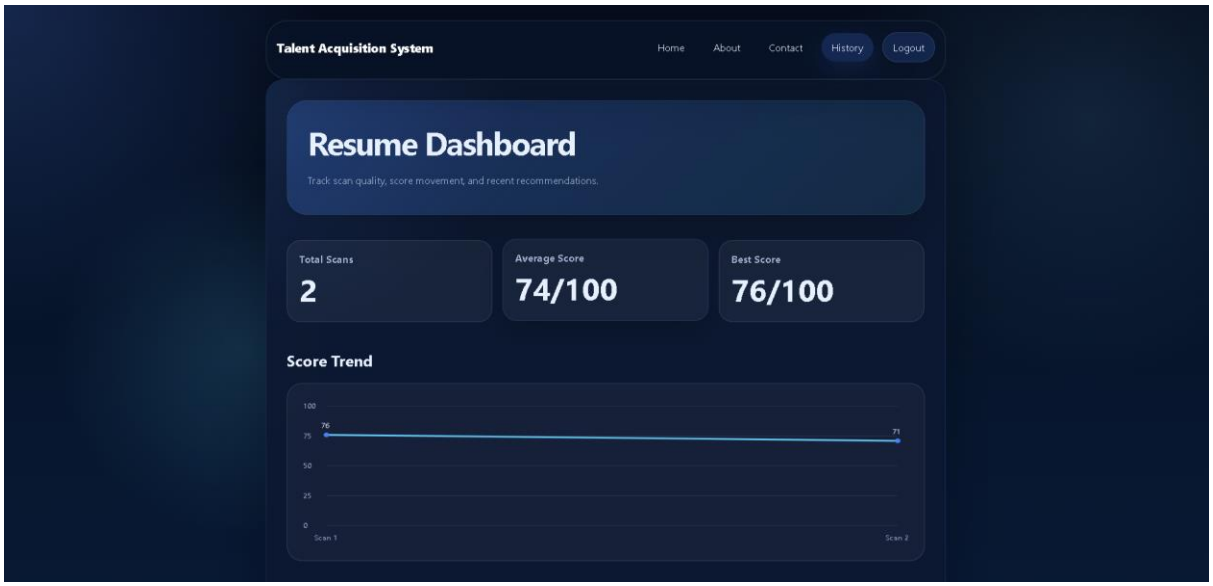
The history feature helps users track resume improvement. A candidate can scan an initial resume, make changes and scan again. The saved reports make it possible to compare progress over time.

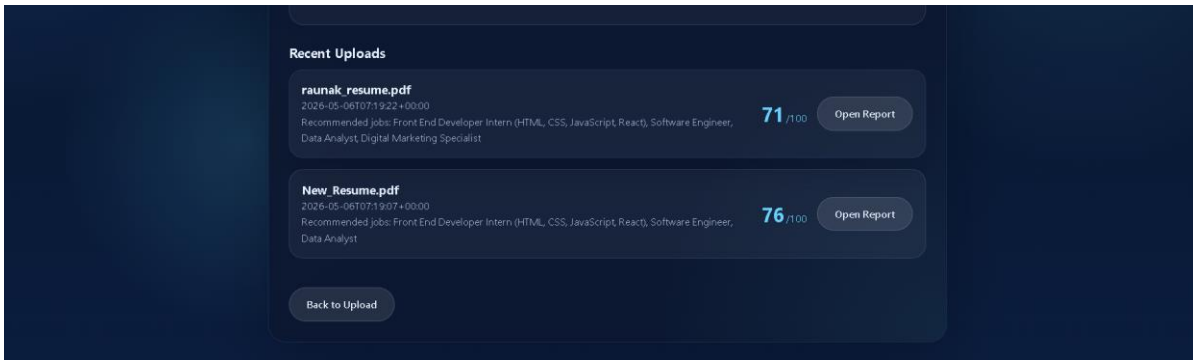
Report details are stored as JSON text where necessary. This approach allows the system to preserve complex nested analysis data while keeping the database schema manageable.

The PDF export feature allows saved report details to be downloaded. This is useful when a student wants to submit the analysis to a mentor or keep a copy for placement preparation.

The export function retrieves the saved report from the database, converts important fields into report lines and generates a simple PDF response. The export is connected to the history detail page.

PDF export improves the practical value of the application because the analysis is not limited to the browser. It can be shared, archived or reviewed later.





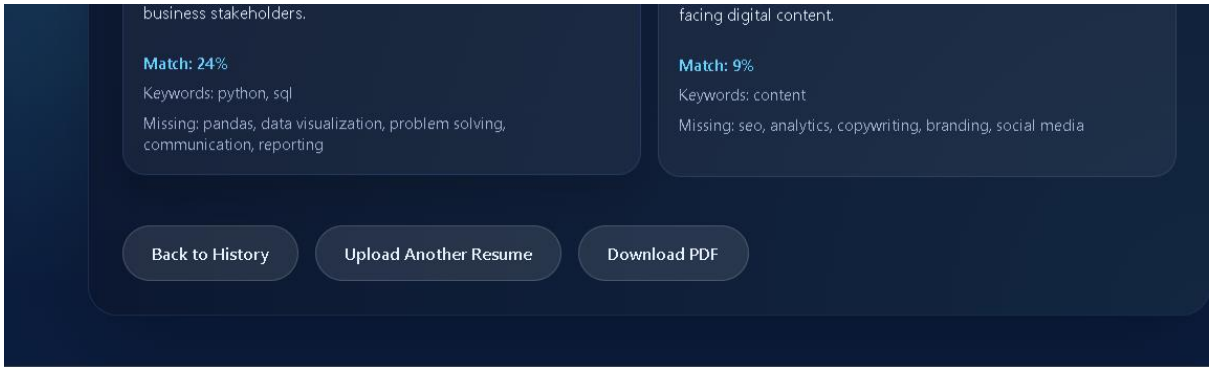


Figure 7: User scan history and saved report details

6.5 ADMIN, CONTACT AND VALIDATION MODULES

The admin module is protected by an admin login requirement. It provides a dashboard for reviewing basic application activity, contact messages and job catalog management.

The admin can add new recommendation roles by entering a title, keywords and description. These entries are saved to the job_data.csv catalog and can be used in future resume matching.

The admin module is important because it gives the system a maintenance path. New job roles can be added without modifying the source code, making the application more flexible for different batches and placement needs.

The contact module allows users to submit their name, email, subject and message. This can be used for support, feedback or placement-related queries. The submitted messages are stored in the contacts table.

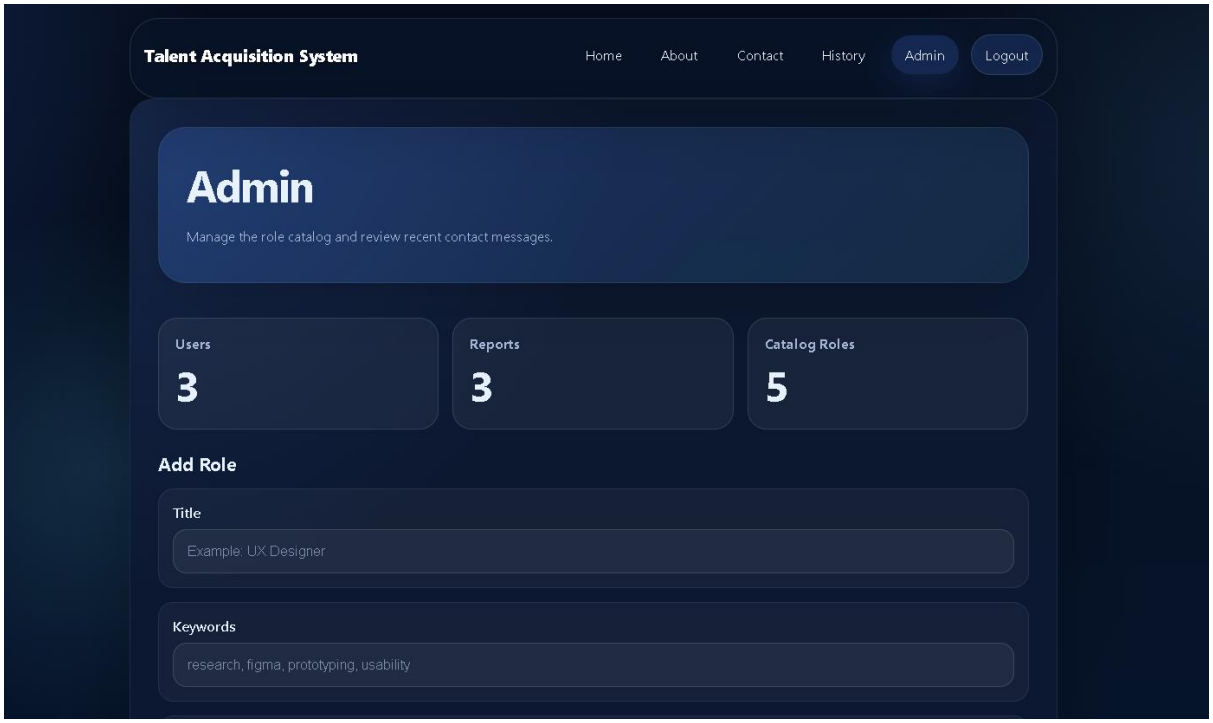
The contact form validates email format before saving. This prevents invalid entries from entering the database and improves the quality of admin review.

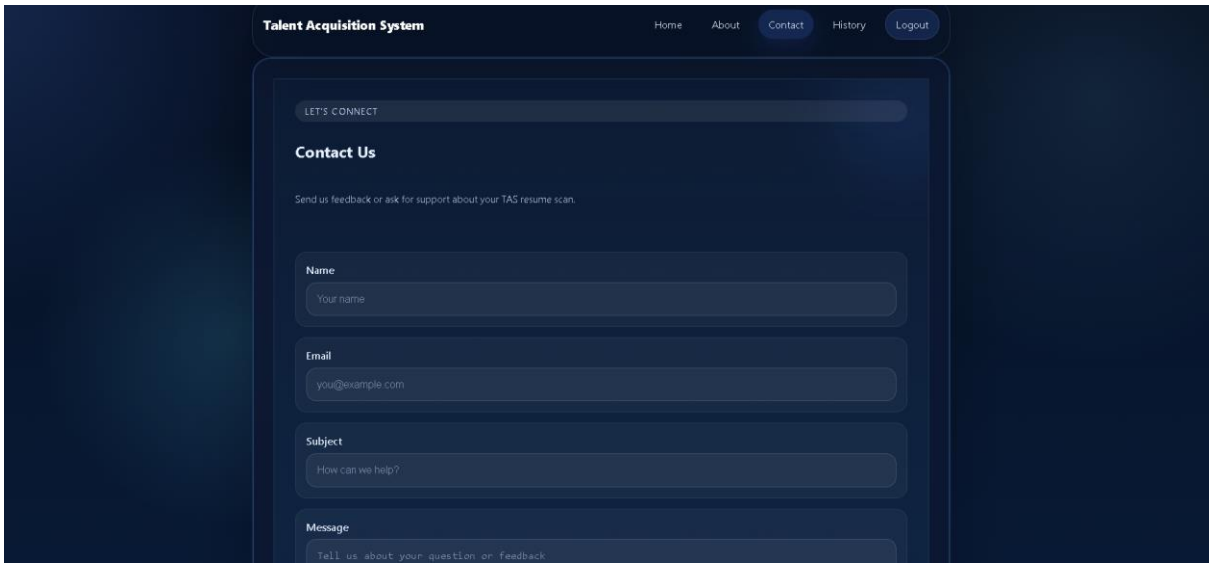
Contact messages are shown in the admin dashboard. This connects user communication with the system's management interface.

The application includes validation at several stages. File upload validation checks missing files, unsupported extensions and mismatched file signatures. Form validation checks required fields and email format.

Error messages are shown using Flask flash messages. This gives users immediate feedback when something goes wrong, such as uploading an unsupported file type or submitting an invalid contact email.

Validation improves both usability and security. It helps users correct mistakes and prevents invalid data from moving through the system.





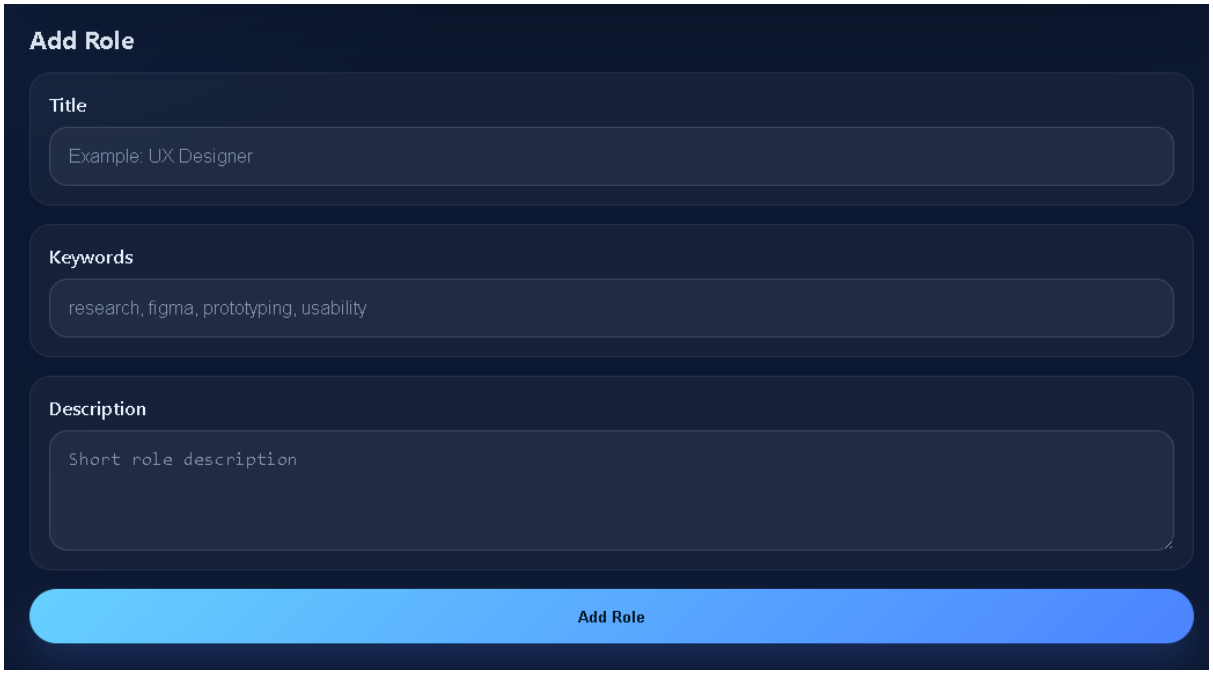


Figure 8: Admin dashboard for catalog and message management

6.6 TESTING STRATEGY AND TEST CASE SUMMARY

Testing is performed using pytest. The testing strategy focuses on critical behavior rather than only checking whether pages load. Important functions and workflows are tested with realistic input.

The tests cover allowed file validation, target job matching, role keyword expansion, skill alias detection, experience estimation, role-adjusted scoring, CSRF blocking, resume analysis, temporary upload deletion, mismatched extension rejection, contact validation, registration, login, history, PDF export and admin catalog update.

This test coverage is useful because the application combines many responsibilities. Automated tests reduce the chance that changes to one feature will break another feature.

The file validation test confirms that only supported resume formats are accepted. The mismatched extension test confirms that the system rejects files whose content does not match the extension. These tests protect the upload workflow.

The authentication test confirms that registration, login, history and PDF export work together. This is important because report ownership depends on the logged-in user. The CSRF test confirms that protected forms reject invalid requests when CSRF is enabled.

The admin catalog test confirms that an admin user can add a role to the catalog. The contact validation test confirms that invalid email addresses are rejected. These tests verify both user-facing and admin-facing behavior.

7. RESULT

The completed system successfully provides a browser-based resume analysis workflow. Users can upload a resume, receive a score, view category-wise explanations, identify matched and missing keywords and see recommended job roles. The result page presents the resume analysis in a structured format that is easier to understand than a single unexplained score.

The system supports PDF, DOCX and TXT formats. It validates both file extension and file signature, helping prevent incorrectly labelled uploads from being processed. Temporary uploads are removed after parsing, which reduces storage of sensitive resume files.

The scoring model produces a total score out of 100 and also provides individual criteria scores. This allows the user to identify specific improvement areas. For example, a resume may score well in education and contact information but poorly in skills or role relevance. This separation makes the feedback actionable.

The target job matching feature compares the resume with a user-provided job title and description. It identifies matched keywords and missing keywords, allowing candidates to improve alignment with the target role. The built-in job catalog provides additional recommendations based on detected skills and resume content.

Registered users can store scan results and view their report history. This feature is useful for tracking improvements across resume versions. The application can also export saved report details as PDF, making it easier to share feedback with mentors or keep records for placement preparation.

The admin dashboard provides basic management features. The administrator can review contact messages, view usage metrics and add job roles to the recommendation catalog. This makes the system adaptable to new placement requirements without changing application code.

Automated tests confirm that major workflows behave as expected. The test suite covers validation, security, analysis, authentication, history, export and admin operations. These tests improve confidence that future changes will not break core behavior.

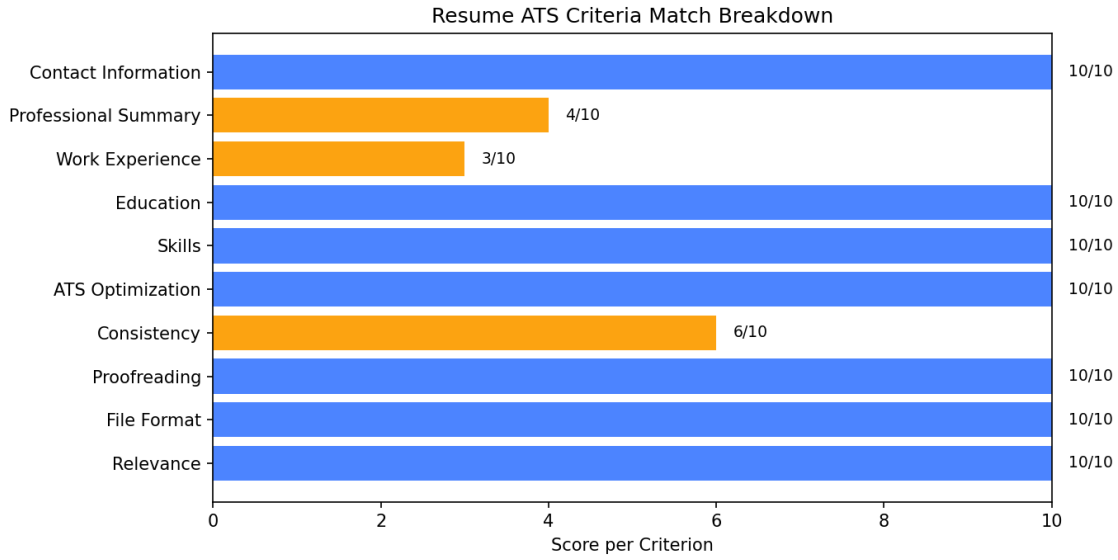


Figure 9: Criteria-wise score visualization generated by the application

7.1 SAMPLE OUTPUT DESCRIPTION

A typical output of the system contains an overall score, criteria-wise score breakdown, matched jobs, target role match, matched keywords, missing keywords, improvement suggestions, analysis steps and rewritten bullet examples. This output gives the user both summary and detail.

The criteria-wise score helps the user understand which section needs improvement. For example, a low proofreading score may indicate spelling issues, while a low relevance score may indicate that the resume does not match the selected job description well.

The matched and missing keyword section is especially useful for job preparation. It gives the user concrete terms to consider adding if they are truthful and relevant to the candidate's experience.

7.2 PERFORMANCE OBSERVATION

The application is lightweight and suitable for local execution. For normal resume files, text extraction and analysis complete quickly because the scoring model is rule-based and does not require expensive external API calls.

SQLite is sufficient for the expected scale of the project. The database stores reports, users and contact messages locally. For a larger deployment, the database could be migrated to PostgreSQL or another server-based DBMS.

The optional OCR feature may take longer because scanned documents require image conversion and character recognition. Since OCR is optional, the main workflow remains efficient for text-based PDF, DOCX and TXT resumes.

7.3 ADVANTAGES OF THE SYSTEM

The system provides immediate resume feedback. This reduces waiting time for students and helps placement teams handle basic review tasks more efficiently. The user can improve the resume and rescan it without depending on manual availability.

The score is transparent because it is divided into criteria. The user can see whether the issue is missing contact information, weak skills, poor relevance or proofreading problems. This makes the feedback actionable.

The system also supports history and export. These features make the application more useful than a one-time checker because users can preserve analysis records and share them for review.

7.4 LIMITATIONS

The system uses rule-based scoring, so it may not understand every resume style or domain-specific phrase. It can detect common patterns and keywords, but it may miss skills written in unusual ways.

The application does not make hiring decisions. It provides resume readiness feedback and role matching suggestions. Human judgment is still required for final recruitment decisions, interview evaluation and candidate selection.

The system is designed for local or small-scale use. For production deployment, additional work would be required, including stronger authentication policies, database migration, privacy review, hosting, logging and backup strategy.

7.5 FUTURE ENHANCEMENTS

Future enhancements can include semantic similarity using transformer-based embeddings. This would allow the system to understand meaning beyond exact keywords. For example, it could match business intelligence with dashboard reporting even when exact words differ.

A placement officer dashboard can be added to view batch-level analytics. This could show average resume scores, common missing skills and improvement trends across students. Such analytics would be useful for training planning.

Other enhancements include email notifications, resume version comparison, richer PDF reports, recruiter-side candidate filtering, cloud deployment, role-specific templates and multilingual resume support.

8. CONCLUSION

The Talent Acquisition System meets the objective of creating a practical resume analysis and job matching web application. It transforms uploaded resume documents into structured insights, applies scoring rules, compares the resume with target roles and provides suggestions that can help candidates improve their resumes.

The project is suitable for academic demonstration because it combines web development, database management, text processing, analytics and testing. It also addresses a real placement-related problem faced by students and job seekers. The scoring model is transparent and explainable, which is valuable in an educational setting.

The system can be enhanced further by adding advanced NLP models, richer resume section extraction, semantic similarity matching, recruiter-side candidate ranking, dashboards for placement officers, email notifications, cloud deployment and stronger privacy controls. Even in its current form, the application provides a useful foundation for resume readiness analysis and placement support.

8.1 SUMMARY OF WORK COMPLETED

The project successfully implements a working resume analysis web application. It includes upload handling, text extraction, scoring, matching, recommendations, user history, PDF export, contact storage and admin catalog management.

The system demonstrates practical data science and analytics by extracting structured indicators from unstructured text. It also demonstrates software engineering through routing, templates, database usage, security controls and testing.

The final project is suitable for academic evaluation because it has a clear problem, defined objectives, working implementation, test coverage and practical relevance to placement preparation.

8.2 LEARNING OUTCOMES

The project helped demonstrate how Python can be used for both web development and text analytics. It shows how Flask routes connect frontend forms with backend processing and how SQLite can store application data.

The project also shows how rule-based analytics can be useful when explainability is important. A transparent scoring model is easier to justify than a black-box model, especially in an academic setting.

Another learning outcome is the importance of validation and testing. File upload applications must be careful with user input, and automated tests help confirm that the main workflows remain reliable.

APPENDIX A: PROJECT FILE STRUCTURE

The project folder contains app.py, templates, static, tests, reports, uploads, job_data.csv, tas_reports.db, requirements.txt and README.md. Each file or folder has a clear responsibility in the application.

The templates folder contains base.html, index.html, result.html, history.html, history_detail.html, login.html, register.html, admin.html, contact.html and about.html. These files define the browser pages rendered by Flask.

The static folder contains style.css, app.js and image assets. The tests folder contains automated test cases. The reports folder contains project documentation files such as this report.

APPENDIX B: IMPORTANT ROUTES

The root route handles the home page and resume analysis form. The analyze route is an alias for posting resume analysis requests. The login, register and logout routes manage user authentication.

The history route displays saved scans for logged-in users. The history detail route displays one saved report, and the export route generates a PDF report. The contact route stores user messages.

The admin route is restricted to the admin user and provides dashboard features. Together these routes cover the main functional requirements of the application.

APPENDIX C: IMPORTANT FUNCTIONS

Important database functions include init_db, create_user, get_user, save_report, get_report_payload and save_contact_submission. These functions create tables, manage accounts and store or retrieve application data.

Important analysis functions include normalize_text, extract_resume_profile, build_criteria_scores, calculate_role_adjusted_score, build_target_job_match, find_applicable_jobs and build_resume_suggestions. These functions produce the main analytical result.

Important security and utility functions include csrf_token, protect_from_csrf, rate_limit, secure temporary upload handling and file signature validation. These functions support safer application behavior.

APPENDIX D: REQUIREMENTS

The project dependencies are Flask, NumPy, pandas, pypdf, python-docx, spaCy, pytest, pytesseract and pdf2image. Some dependencies are essential for the core application, while OCR-related dependencies are optional depending on local system setup.

Flask is required for the web application. pandas and NumPy support data handling and scoring. pypdf and python-docx support resume text extraction. pytest supports automated testing.

The requirements.txt file helps install the needed Python packages in a virtual environment. Using a virtual environment is recommended because it keeps project dependencies separate from the system Python installation.

APPENDIX E: IMPORTANT CODE SNIPPETS

This appendix includes selected code snippets from the application to show the core implementation approach. The complete source code is maintained separately in the project folder, while the report includes only the most relevant parts for academic reference.

The snippets cover database initialization, resume file validation, text extraction, scoring, role matching and catalog update logic. These areas represent the main technical workflow of the Talent Acquisition System.

Database Initialization

```
def init_db() -> None:
    conn = sqlite3.connect(DB_PATH)
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS reports ("
            "id INTEGER PRIMARY KEY AUTOINCREMENT,"
            "user_id INTEGER, filename TEXT, uploaded_at TEXT,"
            "score INTEGER, matched_jobs TEXT, criteria TEXT,"
            "suggestions TEXT, analysis TEXT)"
        )
        conn.commit()
    finally:
        conn.close()
```

Allowed Resume File Validation

```
def allowed_file(filename: str) -> bool:
    return (
        "." in filename
        and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS
    )
```

Resume Text Extraction

```
def extract_resume_text(path: Path, extension: str) -> str:
    if extension == "pdf":
        return extract_text_from_pdf(path)
    if extension == "docx":
        return extract_text_from_docx(path)
    if extension == "txt":
        return path.read_text(encoding="utf-8", errors="ignore")
    return
```

Criteria-wise Score Construction

```
def build_criteria_scores(text: str, extension: str, profile=None, target_text: str
= "") -> dict:
    return {
        "Contact Information": detect_contact_info(text),
        "Professional Summary": detect_summary(text),
        "Work Experience": detect_experience(text, profile),
        "Education": detect_education(text),
        "Skills": detect_skills(text, profile),
        "TAS Optimization": detect_tas_optimization(text),
        "Consistency": detect_consistency(text),
```

```
        "Proofreading": detect_proofreading(text),
        "File Format": 100 if extension in ALLOWED_EXTENSIONS else 20,
        "Relevance": detect_relevance(text, target_text),
    }
```

Role-adjusted Final Score

```
def calculate_role_adjusted_score(criteria_scores: dict, target_title: str = "") ->
int:
    weights = scoring_weights_for_role(target_title)
    total = sum(criteria_scores.get(name, 0) * weight for name, weight in
weights.items())
    return int(np.clip(round(total), 0, 100))
```

Job Recommendation Match

```
def find_applicable_jobs(text: str, profile: dict | None = None, target_job: dict |
None = None) -> list:
    jobs_df = load_job_data()
    resume_terms = build_weighted_terms(text)
    recommendations = []
    if target_job:
        recommendations.append(target_job)
    for _, row in jobs_df.iterrows():
        keywords = [item.strip() for item in row["keywords"].split(",")]
        job_text = f"{row['title']} {row.get('description', '')} {'
'.join(keywords)}"
        similarity = weighted_cosine_similarity(resume_terms,
build_weighted_terms(job_text))
        if similarity > 0.18:
            recommendations.append({"title": row["title"], "match_score":
round(similarity * 100)})
    return recommendations
```

Admin Catalog Update

```
new_row = pd.DataFrame([{
    "title": title,
    "keywords": keywords,
    "description": description,
}])
jobs_df = pd.concat([jobs_df, new_row], ignore_index=True)
jobs_df.to_csv(JOB_DATA_PATH, index=False)
```

REFERENCES

Flask Documentation, Pallets Projects, https://flask.palletsprojects.com/

Python Documentation, Python Software Foundation, https://docs.python.org/

SQLite Documentation, SQLite Consortium, https://www.sqlite.org/docs.html

pandas Documentation, NumFOCUS, https://pandas.pydata.org/docs/

NumPy Documentation, NumPy Developers, https://numpy.org/doc/

python-docx Documentation, https://python-docx.readthedocs.io/

pypdf Documentation, https://pypdf.readthedocs.io/

spaCy Documentation, Explosion AI, https://spacy.io/usage

Werkzeug Documentation, Pallets Projects, https://werkzeug.palletsprojects.com/

pytest Documentation, https://docs.pytest.org/